persistence layer and is integrated with Grafana, Prometheus, and Loki. It supports Jaeger, Zipkin, and OpenTelemetry ingestion.

Now, let's move on to commercial offerings, which are, in contrast to open source offerings, very richly equipped. This starts at correlation features and ranges all the way to sampling and integrations.

5.4.3 Commercial offerings for trace backends

Originally called *application performance monitoring* (APM) vendors, nowadays, the tracing aspect (or *observability solution*) is often highlighted. We will cover these offerings in detail in chapter 6.

While distributed tracing hasn't yet reached the same level of adoption as metrics and logs, it's increasingly important for cloud-native setups. Even if you don't have a short-term strategy around it, consider testing it in smaller-scoped projects or products. Now, we'll shift gears to discussing a class of backends that helps address the aforementioned challenge of cardinality explosion: columnar data stores and formats.

5.5 Columnar data stores

There are many so-called columnar storage–based engines or simply data stores that you can use as backends for telemetry data. *Columnar* refers to the way the data is laid out in memory or on disk, boosting query performance.

Row-oriented databases, such as MySQL and PostgreSQL, keep all of the data associated with a record in close proximity in memory (or on disk). This means, in general, row-oriented DBs are a good fit for online transaction processing (OLTP) use cases, such as e-commerce, in which you have in-place updates of records. They are optimized for reading and writing rows, which is a great fit when you have many individual transactions with a potential for low data volume, such as is the case when inserting or updating single records in an online web application (e.g., putting a product into a shopping basket, following someone on a social media site, and so on).

Column-oriented or columnar data stores (see Mike Stonebraker et. al., "C-Store: A Column-oriented DBMS." Proceedings of the 31st VLDB Conference, Trondheim, Norway, 2005; see also https://people.brandeis.edu/~nga/papers/VLDB05.pdf) store the data of columns next to each other and are typically a good choice for so-called online analytical processing (OLAP) use cases. These are cases where you have few transactions but are dealing with a high volume of queries. The write path is the ingest of, say, time series data, and the read path is, for example, an ad hoc query over a time range. These columnar data stores are, hence, well suited for historical (time series) data where we want to extract actionable insights, typically in an interactive manner, from large datasets (many billions of rows and, potentially, petabytes of data) or, in other words, observability use cases. Let's have a closer look at this: figure 5.5 shows row-oriented versus column-oriented storage strategies.

Starting with the logical layout, a tabular view on columns (vertical) and rows (horizontal), a particular value has the coordinates `CiRj` (e.g., the value of the third